



(<https://www.nvidia.com/dli>)

使用 Numba 在 Python 中编写自定义 CUDA 核函数

本节中，我们会进一步了解 CUDA 编程模型如何调度并行任务，并将以此为基础来编写自定义 CUDA 核函数，即在 NVIDIA GPU 上并行执行的函数。相较于仅使用 `@vectorize` 装饰通用函数 (ufunc) 的情况，利用 CUDA 编程模型编写自定义 CUDA 核函数需耗费更多工作。然而，自定义 CUDA 核函数能在 ufunc 无法发挥作用的领域实现并行计算，且其提供的灵活性亦能带来超强性能。

如您有兴趣做进一步研究，请参阅本节包含的三个附录：协助您进行 GPU 编程的各类调试技术、CUDA 编程参考链接，以及在 GPU 上生成 Numba 支持的随机数。

目标

完成本节内容的学习后，您将能够：

- 在 Python 中编写自定义 CUDA 核函数，并使用执行配置启动这些核函数。
- 利用网格跨度循环并行处理庞大的数据集，以及利用内存合并。
- 在并行执行工作时，使用原子操作避免竞争条件。

自定义核函数的需求

Ufunc 十分精妙，对于要在数据上执行的任何元素级标量运算，它都可能是一款合适的工具。

想必大家也很清楚，在解决多种问题时，为数据集的每个元素应用相同的函数是行不通的。这类示例包括任何需访问数据结构的多个元素才能计算其输出的问题（如 Stencil 算法），或任何无法通过输入值到输出值的映射来表示的问题（如归约）。此类问题有许多在本质上仍是可并行的，但却无法用 ufunc 表示。

尽管编写自定义 CUDA 核函数要比编写 GPU 加速的 ufunc 更具挑战性，但此举能为开发者提供巨大的灵活性，以便在 GPU 上并行运行各类函数。此外，在开启本节和下一节学习之旅时，您将了解到，自定义 CUDA 核函数还允许开发者显式地使用 CUDA 的线程层次结构，实现对如何并行执行进行的精细控制。

虽然整个过程均在 Python 中进行，但在使用 Numba 编写 CUDA 核函数时，您会很容易联想到用 CUDA C/C++ 编写这些函数的具体方法。如果您熟悉用 CUDA C/C++ 编程，那么您会迅速掌握使用 Numba 在 Python 中编写自定义核函数的要领。若您是 CUDA 初学者，需要或希望使用 C/C++ 开发 CUDA 应用，乃至想研究在互联网上的丰富的 CUDA C/C++ 代码资源，那么本节的学习将能让您大有所获。

CUDA 核函数简介

在用 CUDA 编程时，开发者会为 GPU 编写名为核函数的函数，且该函数可在多个 GPU 核心上以并行线程执行（在 CUDA 中即为启动）。启动核函数时，程序员会使用一种名为执行配置（亦称为“启动配置”）的特殊语法来描述并行执行的配置。

以下幻灯片（执行下方单元后即会显示）大体讲解了如何创建 CUDA 核函数，从而在 GPU 设备上并行处理大型数据集。请浏览所有幻灯片，然后使用其提供的理念，开始编写和执行您自己的自定义 CUDA 核函数。

```
In [ ]: from IPython.display import IFrame
        IFrame('https://view.officeapps.live.com/op/view.aspx?src=https://developer.download.nvidia.com/training/courses')
```

第一个 CUDA 核函数

我们首先来看一个非常简单的具体示例：为一维 NumPy 数组重写加法函数。我们会使用 `numba.cuda.jit` 装饰器编译 CUDA 核函数。请勿将 `numba.cuda.jit` 与您刚学习的 `numba.jit` 装饰器混淆，后者的作用是 CPU 优化函数。

我们首先会展示一个非常简单的示例，用以强调一些基本语法。值得一提的是，此函数实际可写成 ufunc，而我们在此选用它是为了重点学习自定义函数的语法。在后面我们将继续介绍更适合写成自定义核函数的函数。请务必仔细阅读注释，以便了解一些重要的代码信息。

```
In [ ]: from numba import cuda

# Note the use of an `out` array. CUDA kernels written with `@cuda.jit` do not return values,
# just like their C counterparts. Also, no explicit type signature is required with @cuda.jit
@cuda.jit
def add_kernel(x, y, out):

    # The actual values of the following CUDA-provided variables for thread and block indices,
    # like function parameters, are not known until the kernel is launched.

    # This calculation gives a unique thread index within the entire grid (see the slides above for more)
    idx = cuda.grid(1)          # 1 = one dimensional thread grid, returns a single value.
                                # This Numba-provided convenience function is equivalent to
                                # `cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x`

    # This thread will do the work on the data element with the same index as its own
    # unique index within the grid.
    out[idx] = x[idx] + y[idx]
```

```
In [ ]: import numpy as np

n = 4096
x = np.arange(n).astype(np.int32) # [0...4095] on the host
y = np.ones_like(x)               # [1...1] on the host

d_x = cuda.to_device(x) # Copy of x on the device
d_y = cuda.to_device(y) # Copy of y on the device
d_out = cuda.device_array_like(d_x) # Like np.array_like, but for device arrays

# Because of how we wrote the kernel above, we need to have a 1 thread to one data element mapping,
# therefore we define the number of threads in the grid (128*32) to equal n (4096).
threads_per_block = 128
blocks_per_grid = 32
```

```
In [ ]: add_kernel[blocks_per_grid, threads_per_block](d_x, d_y, d_out)
        cuda.synchronize()
        print(d_out.copy_to_host()) # Should be [1...4096]
```

练习：修改代码

对上方代码做如下细微修改，并查看该操作会对代码执行产生何种影响。运行代码前，请先对结果作出合理猜测：

- 减少 `threads_per_block` 变量
- 减少 `blocks_per_grid` 变量
- 增加 `threads_per_block` 或 `blocks_per_grid` 变量
- 删除 `cuda.synchronize()` 调用，或将其改为注释

结果

在上方示例中，由于核函数的编写使每个线程仅处理一个数据元素，因此网格中的线程数必须与数据元素的数量相等。

在**减少网格中的线程数**（通过减少块数或每块的线程数）后，部分数据元素的工作便无法完成，因此我们可以在输出中看到，`d_out` 数组末尾的元素并未获得任何值。如果您通过减少每块的线程数来编辑执行配置，则 `d_out` 数组中实际仍有其他元素无法得到处理。

增加网格大小实际上会导致内存访问超界错误。在本节的后面，您将学习如何使用 `cuda-memcheck` 找出此错误，并对其进行调试。

您可能会猜测，**删除同步点**后将会产生打印信息，显示出并未完成任何工作，或仅完成了少量工作。这种猜测合情合理，因为若缺乏同步点，CPU 将在 GPU 处理期间异步执行工作。此处要学习的细节是，内存复制包含隐式同步，因此调用 `cuda.synchronize` 就没有必要了。

练习：使 CPU 函数作为自定义 CUDA 核函数以实现加速

下方是一个可用作 CPU ufunc 的 CPU 标量函数 `square_device`。您的任务是重构该函数，使其作为经 `@cuda.jit` 装饰器装饰的 CUDA 核函数运行。

您可能会认为，使用 `@vectorize` 将能更轻松地在设备上运行此函数，而事实的确如此。但在继续学习更复杂的实际示例前，这种场景可让您有机会使用我们所介绍的全部语法。

在本次练习中，您需要：

- 将 `square_device` 的定义重构为 CUDA 核函数，使单线程只在单个数据元素上完成计算任务。
- 将下方的 `d_a` 和 `d_out` 数组重构为 CUDA 设备数组。
- 修改 `blocks` 和 `threads` 变量，以适应给定的 `n` 的值。
- 重写对 `square_device` 的调用，使它做为包含执行配置的核函数被启动。

只有在您成功实现以上操作后，以下断言测试才会成功。如您遇到问题，请随时参阅 [此解决方案](#) ([../edit/solutions/square_device_solution.py](#))。

```
In [ ]: # Refactor to be a CUDA kernel doing one thread's work.
# Don't forget that when using `@cuda.jit`, you must provide an output array as no value will be returned.
def square_device(a):
    return a**2

In [ ]: # Leave the values in this cell fixed for this exercise
n = 4096

a = np.arange(n)
out = a**2 # `out` will only be used for testing below

In [ ]: d_a = a # TODO make `d_a` a device array
d_out = np.zeros_like(a) # TODO: make d_out a device array

# TODO: Update the execution configuration for the amount of work needed
blocks = 0
threads = 0

# TODO: Launch as a kernel with an appropriate execution configuration
d_out = square_device(d_a)

In [ ]: from numpy import testing
testing.assert_almost_equal(d_out, out)
```

关于隐藏延迟与选择执行配置的说明

对于支持 CUDA 的 NVIDIA GPU 而言，其每个晶片上均包含数个 [流多处理器 \(SM\)](https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#hardware-implementation)，并附带 DRAM。SM 包含执行核函数代码所需的所有资源，并且包括多个 CUDA 核心。启动核函数时，每个线程块只分给一个 SM，亦有可能多个线程块分给一个 SM。SM 会将线程块进一步细分为每32个线程一个单位（称为 **warp**），而接收并执行并行指令的正是这些 warps。

当一条指令需要多个时钟周期才能完成（或以CUDA的说法是**到期**）时，如果仍有其它的 warps 等待接收新指令，则 SM 便能继续做有意义的工作。由于 SM 上的寄存器堆非常庞大，因而在转向一个新的 warp 发布指令时，SM 不会因改变工作的上下文环境而造成时间损失。简言之，只要有其它待做的工作，SM 便会一直执行有意义的工作而将操作延迟隐藏起来。

因此，对于充分利用 GPU 的潜力并进而编写出高性能的加速应用程序而言，最重要的是必须为 SM 提供足够数量的 warps，使 SM 能够隐藏延迟。而实现这一目的的最简单的方法便是使用足够大的网格与线程块来执行核函数。

确定CUDA线程网格的最佳大小是一个复杂的问题，取决于算法和特定的GPU的[计算能力](https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#compute-capabilities)。不过，以下是几条粗略的启发式规则，遵循它们可以很好地帮助我们入门：

- 一个线程块所含的线程数应为 32（warp）的倍数，每个线程块通常包含 128 至 512 个线程。
- 网格大小应确保能够充分利用 GPU 的全部潜能。学习伊始，建议您在 GPU 上启动的网格里的块数是 SM 数的2至4倍。使用 20 至 100 个线程块通常是一个适合的起点。
- CUDA 核函数的启动开销的确会随块数而增长，因此在输入的数据规模非常庞大时，建议您不要启动线程数与输入元素数相等的网格，以免产生大量的线程块。相反，我们可以改用另一种模式。下面，就让我们着重探讨一下如何通过该模式处理大规模的输入数据。

使用跨网格循环处理超大型数据集

以下幻灯片概述了一项名为**跨网格循环**的技术。该技术可以创建灵活的核函数，让每个线程均能处理多个数据元素，因而能够满足大型数据集的处理需求。请执行下方单元，以加载幻灯片。

```
In [1]: from IPython.display import IFrame
        IFrame('https://view.officeapps.live.com/op/view.aspx?src=https://developer.download.nvidia.com/training/courses')

Out[1]:
```

```
In [ ]:
```

第一个跨网格循环

通过重构上面的 `add_kernel` 函数，我们便可利用并启动跨网格循环，让其灵活处理规模更大的数据集，同时还能享受全局内存合并带来的益处，即使并行线程访问连续数据块中的内存，进而协助 GPU 减少内存操作的总次数：

```
In [2]: from numba import cuda

        @cuda.jit
        def add_kernel(x, y, out):

            start = cuda.grid(1)

            # This calculation gives the total number of threads in the entire grid
            stride = cuda.gridsize(1) # 1 = one dimensional thread grid, returns a single value.
                                     # This Numba-provided convenience function is equivalent to
                                     # `cuda.blockDim.x * cuda.gridDim.x`

            # This thread will start work at the data element index equal to that of its own
            # unique index in the grid, and then, will stride the number of threads in the grid each
            # iteration so long as it has not stepped out of the data's bounds. In this way, each
            # thread may work on more than one data element, and together, all threads will work on
            # every data element.
            for i in range(start, x.shape[0], stride):
                # Assuming x and y inputs are same length
                out[i] = x[i] + y[i]
```

```
In [ ]:
```

```
In [3]: import numpy as np

        n = 100000 # This is far more elements than threads in our grid
        x = np.arange(n).astype(np.int32)
        y = np.ones_like(x)

        d_x = cuda.to_device(x)
        d_y = cuda.to_device(y)
        d_out = cuda.device_array_like(d_x)

        threads_per_block = 128
        blocks_per_grid = 30
```

```
In [ ]:
```

```
In [4]: add_kernel[blocks_per_grid, threads_per_block](d_x, d_y, d_out)
        print(d_out.copy_to_host()) # Remember, memory copy carries implicit synchronization

        [ 1      2      3 ... 99998 99999 100000]
```

```
In [ ]:
```

练习：实现跨网格循环

对下方的 CPU 标量函数 `hypot_stride` 进行重构，使其能使用跨网格循环并作为 CUDA 核函数运行。如您遇到问题，请随时参阅 [此解决方案](#) ([./edit/solutions/hypot_stride_solution.py](#))。

```
In [5]: from math import hypot

        def hypot_stride(a, b, c):
            c = hypot(a, b)
```

```
In [ ]:
```

```
In [ ]: # You do not need to modify the contents in this cell
        n = 1000000
        a = np.random.uniform(-12, 12, n).astype(np.float32)
        b = np.random.uniform(-12, 12, n).astype(np.float32)
        d_a = cuda.to_device(a)
        d_b = cuda.to_device(b)
        d_c = cuda.device_array_like(d_b)

        blocks = 128
        threads_per_block = 64

        hypot_stride[blocks, threads_per_block](d_a, d_b, d_c)
```

```
In [ ]: from numpy import testing
        # This assertion will fail until you successfully implement the hypot_stride kernel above
        testing.assert_almost_equal(np.hypot(a,b), d_c.copy_to_host(), decimal=5)
```

为核函数计时

让我们花点时间为 `hypot_stride` 核函数进行性能计时。若您无法成功实现此操作，请在计时前复制并执行[此解决方案](#) ([./edit/solutions/hypot_stride_solution.py](#))。

CPU 基准

首先我们用 `np.hypot` 获取基准：

```
In [ ]: %timeit np.hypot(a, b)
```

基于 CPU 的 Numba

接下来，让我们看一下 CPU 优化后的版本：

```
In [ ]: from numba import jit

        @jit
        def numba_hypot(a, b):
            return np.hypot(a, b)
```

```
In [ ]: %timeit numba_hypot(a, b)
```

设备上的单线程

我们在仅拥有单线程的网格中启动核函数。我们将在此使用 `%time`，这将仅运行一次语句，可确保我们的测量结果不受 CUDA 核函数队列的有限深度的影响。我们还将添加 `cuda.synchronize`，以确保在核函数运行完毕之前，我们不会因将控制权交还给 CPU（计时器所处位置）而得到任何错误时间：

```
In [ ]: %time hypot_stride[1, 1](d_a, d_b, d_c); cuda.synchronize()
```

此方式的执行速度甚至比基准 CPU 还要慢，希望不会令您大吃一惊。

设备上的并行

```
In [ ]: %time hypot_stride[128, 64](d_a, d_b, d_c); cuda.synchronize()
```

速度更胜以往！

原子操作与避免竞争条件

与众多通用型并行执行框架类似，CUDA 也可能让您的代码里产生竞争条件。当一个线程读取或写入可能由另一个独立线程修改的内存位置时，就会出现 CUDA 中的竞争条件。您通常需要担心以下问题：

- 先写后读的风险：在某个线程正向内存位置写入数据时，另一个线程可能正在进行读取。
- 写后再写的风险：两个线程同时向同一内存位置写入数据，而在核函数运行完毕后，仅有一个写入是可见的。

避免这两种风险的常见策略是组织 CUDA 核函数算法，使每个线程对输出数组元素的唯一子集担负专属责任；并且/或者，不在单一核函数的调用中同时使用相同数组用于输入和输出。（视需要，您也可在迭代算法中使用双缓冲策略，并在每次迭代中交换输入和输出数组。）

但在很多情况下，不同的线程都需要与结果进行结合。举个简单的例子：“每个线程对一个全局计数器里的值增1”。如要在核函数中实现此操作，则每个线程需要：

1. 读取全局计数器 `counter` 的当前值。
2. 计算 `counter + 1`。
3. 将该值写回全局内存。

然而，您无法保证另一个线程不会在第 1 步和第 3 步之间更改全局计数器。为解决此问题，CUDA 提供了**原子操作**，此操作将能通过一个不可分步骤来读取、修改和更新内存位置。Numba 支持其中几类函数，[详情请见此处](http://numba.pydata.org/numba-doc/dev/cuda/intrinsics.html#supported-atomic-operations)（<http://numba.pydata.org/numba-doc/dev/cuda/intrinsics.html#supported-atomic-operations>）。

下面，让我们编写自己的线程计数器核函数：

```
In [6]: @cuda.jit
def thread_counter_race_condition(global_counter):
    global_counter[0] += 1 # This is bad

@cuda.jit
def thread_counter_safe(global_counter):
    cuda.atomic.add(global_counter, 0, 1) # Safely add 1 to offset 0 in global_counter array
```

```
In [ ]:
```

```
In [ ]: # This gets the wrong answer
global_counter = cuda.to_device(np.array([0], dtype=np.int32))
thread_counter_race_condition[64, 64](global_counter)

print('Should be %d:' % (64*64), global_counter.copy_to_host())
```

```
In [ ]: # This works correctly
global_counter = cuda.to_device(np.array([0], dtype=np.int32))
thread_counter_safe[64, 64](global_counter)

print('Should be %d:' % (64*64), global_counter.copy_to_host())
```

评估

下面的练习将用到您目前所学的全部知识。不同于之前的练习，本次练习不提供任何解决方案代码，且您还需采取一些其他步骤来“运行评估”，以获得操作分数。**请仔细阅读说明后再开始工作，确保以最大机率成功完成本次评估。**

如何运行评估

请执行以下步骤完成评估：

1. 按照以下说明，像平常练习一样运行下方单元。
2. 若您对自己的执行效果甚感满意，请按照下方说明，将代码复制粘贴到所关联的源代码文件中。代码粘贴完成后，务必保存文件。
3. 返回至您用来启动此笔记本的浏览器选项卡，然后点击“**Assess**”（评估）按钮。几秒后会生成分数，同时还将提供一条实用信息。

您可视需要点击“**Assess**”（评估）按钮，次数不限。如果您首次未获通过，也不必担心，只需对代码作出其他修改并重复以上三个步骤，即可再次进行评估。祝您好运！

Home **Course** Progress Instructor

Course > Fundamentals of Accelerated Computing with CUDA Python > Custom Kernels and Memory Management for CUDA Python with Numba > Custom CUDA Kernels in Python with Numba

< Previous  Next >

Custom CUDA Kernels in Python with Numba VIEW UNIT IN STUDIO

[Bookmark this page](#)

Click here to earn credit for your work



 Powered by: 

2 : 12 : 34 REMAINING TIME  LAUNCH TASK  STOP TASK  ASSESS TASK

After a view minutes a LAUNCH TASK play button will appear which you can click on to start the interactive coding environment.

编写加速直方图核函数

本次评估中，您将创建加速直方图核函数。在此过程中，您需使用输入数据数组、范围、一定数量的累计箱，并需计算每个累计箱中落入的输入数据元素数量。下方为 CPU 实现的有效直方图实例，您可以此为例来开展自己的工作：

```
In [7]: def cpu_histogram(x, xmin, xmax, histogram_out):
        '''Increment bin counts in histogram_out, given histogram range [xmin, xmax).'''
        # Note that we don't have to pass in nbins explicitly, because the size of histogram_out determines it
        nbins = histogram_out.shape[0]
        bin_width = (xmax - xmin) / nbins

        # This is a very slow way to do this with NumPy, but looks similar to what you will do on the GPU
        for element in x:
            bin_number = np.int32((element - xmin)/bin_width)
            if bin_number >= 0 and bin_number < histogram_out.shape[0]:
                # only increment if in range
                histogram_out[bin_number] += 1
```

In []:

```
In [8]: x = np.random.normal(size=10000, loc=0, scale=1).astype(np.float32)
        xmin = np.float32(-4.0)
        xmax = np.float32(4.0)
        histogram_out = np.zeros(shape=10, dtype=np.int32)

        cpu_histogram(x, xmin, xmax, histogram_out)

        histogram_out
```

```
Out[8]: array([ 13,   79,  447, 1529, 2907, 2924, 1523,  489,   84,    5],
              dtype=int32)
```

In []:

请使用跨网格循环和原子操作，并通过下方单元执行您的解决方案。在作出任何修改后，请将此单元的内容粘贴至 [assessment/histogram.py](#) ([./edit/assessment/histogram.py](#)) 并保存，之后再运行评估。

```
In [12]: @cuda.jit
def cuda_histogram(x, xmin, xmax, histogram_out):
    '''Increment bin counts in histogram_out, given histogram range [xmin, xmax).'''
    start = cuda.grid(1)
    stride = cuda.gridsize(1)
    nbins = histogram_out.shape[0]
    bin_width = (xmax - xmin) / nbins
    for i in range(start, x.shape[0], stride):
        bin_number = np.int32((x[i] - xmin)/bin_width)
        if bin_number >= 0 and bin_number < histogram_out.shape[0]:
            # only increment if in range
            # histogram_out[bin_number] += 1
            cuda.atomic.add(histogram_out, bin_number, 1)

    # pass # Replace this with your implementation
```

```
In [ ]:
```

```
In [13]: d_x = cuda.to_device(x)
d_histogram_out = cuda.device_array_like(histogram_out)

blocks = 128
threads_per_block = 64

cuda_histogram[blocks, threads_per_block](d_x, xmin, xmax, d_histogram_out)
```

```
In [ ]:
```

```
In [14]: # This assertion will fail until you correctly implement `cuda_histogram`
np.testing.assert_array_almost_equal(d_histogram_out.copy_to_host(), histogram_out, decimal=2)
```

```
In [ ]:
```

总结

在本节中，您已学习如何：

- 在 Python 中编写自定义 CUDA 核函数，并使用执行配置启动这些核函数。
- 利用跨网格循环以及内存合并，并行处理庞大的数据集。
- 在并行执行工作时，使用原子操作避免竞争条件。

下载内容

如要下载此笔记本的内容，请执行以下单元，然后点击下方的下载链接。注意：由于笔记本中的部分文件路径链接是专为我们的平台量身设计，若您在本地的 Jupyter 服务器上运行此笔记本，这些链接可能不是有效的。不过，您仍可通过 Jupyter 文件导航器导航至这些文件。

```
In [ ]: !tar -zcvf section2.tar.gz .
```

[下载本节文件。\(files/section2.tar.gz\)](#)

附录：故障排除和调试

有关终端的注释

调试是编程的重要组成部分。很遗憾，由于各种原因，我们很难直接在 Jupyter Notebook 中调试 CUDA 核函数。基于此，此笔记本将使用 shell 执行 Jupyter Notebook 单元，从而显示终端命令。这些 shell 命令会出现在笔记本单元中，且命令行前会加上 !。在应用此笔记本中介绍的调试方法后，您便可能直接在终端中运行命令。

打印

常见的调试策略是打印至控制台。Numba 支持从 CUDA 核函数打印，但存在一些限制。注意，Jupyter 不会捕获从 CUDA 核函数打印的输出，因此您需使用可从终端运行的脚本进行调试。

下面让我们来看看出错的 CUDA 核函数：

```
In [ ]: ! cat debug/ex1.py
```

在对直方图（50 个值）运行这段代码后，我们发现直方图并未得到 50 个条目：


```
In [ ]: ! python debug/ex1.py
```

(您可能已经发现该错误, 但我们先假装不知道原因。)

我们假设, 可能是由于累计箱计算有误, 才导致许多直方图条目超出范围。让我们在 if 语句周围添加一些打印信息, 看看会出现何种情况:

```
In [ ]: ! cat debug/ex1a.py
```

此核函数将打印其计算出的每个值和累计箱数量。从其中一个打印语句中, 我们可以看到 print 支持常量字符串和标量值:

```
print('in range', x[i], bin_number)
```

但其不支持字符串替换 (使用 C 语言 printf 语法或更新的 format() 语法得出)。如果运行下方脚本, 我们会看到:

```
In [ ]: ! python debug/ex1a.py
```

扫描此输出后, 我们发现 50 个值均应在范围之内。很明显, 肯定有某种竞争条件正在更新直方图。而事实上, 罪魁祸首应为如下代码行:

```
histogram_out[bin_number] += 1
```

它应该是 (您可能已在上一练习中见到):

```
cuda.atomic.add(histogram_out, bin_number, 1)
```

CUDA 模拟器

在 CUDA 的早期发展阶段, nvcc 具有一种“模拟器”模式, 该模式可在 CPU 上执行 CUDA 代码, 进而完成调试。但在创建 cuda-gdb 后, 后续 CUDA 版本中便摒弃了该功能。由于没有同时适用于 CUDA 和 Python 的调试器, 因此 Numba 在内部加入了“CUDA 模拟器”, 以便能够在主机 CPU 上使用 Python 解释器运行 CUDA 代码。借助该模拟器, 您将能使用编译所不支持的 Python 模块和函数, 以调试代码的逻辑。

下面是一个常见用例, 其作用是在 CUDA 核函数的一个线程内启动 Python 调试器:

```
import numpy as np

from numba import cuda

@cuda.jit
def histogram(x, xmin, xmax, histogram_out):
    nbins = histogram_out.shape[0]
    bin_width = (xmax - xmin) / nbins

    start = cuda.grid(1)
    stride = cuda.gridsize(1)

    ### DEBUG FIRST THREAD
    if start == 0:
        from pdb import set_trace; set_trace()
    ###

    for i in range(start, x.shape[0], stride):
        bin_number = np.int32((x[i] + xmin)/bin_width)

        if bin_number >= 0 and bin_number < histogram_out.shape[0]:
            cuda.atomic.add(histogram_out, bin_number, 1)

    x = np.random.normal(size=50, loc=0, scale=1).astype(np.float32)
    xmin = np.float32(-4.0)
    xmax = np.float32(4.0)
    histogram_out = np.zeros(shape=10, dtype=np.int32)

    histogram[64, 64](x, xmin, xmax, histogram_out)

    print('input count:', x.shape[0])
    print('histogram:', histogram_out)
    print('count:', histogram_out.sum())
```

这段代码可实现如下所示的调试会话:

```
(gtc2017) 0179-sseibert:gtc2017-numba sseibert$ NUMBA_ENABLE_CUDASIM=1 python debug/ex2.py
> /Users/sseibert/continuum/conferences/gtc2017-numba/debug/ex2.py(18)histogram()
-> for i in range(start, x.shape[0], stride):
(Pdb) n
> /Users/sseibert/continuum/conferences/gtc2017-numba/debug/ex2.py(19)histogram()
-> bin_number = np.int32((x[i] + xmin)/bin_width)
(Pdb) n
> /Users/sseibert/continuum/conferences/gtc2017-numba/debug/ex2.py(21)histogram()
-> if bin_number >= 0 and bin_number < histogram_out.shape[0]:
(Pdb) p bin_number, x[i]
(-6, -1.4435024)
(Pdb) p x[i], xmin, bin_width
(-1.4435024, -4.0, 0.8000000000000004)
(Pdb) p (x[i] - xmin) / bin_width
```

CUDA Memcheck

当 CUDA 核函数的内存访问无效时，另一个常见错误便会出现，其诱因通常是数组越界。NVIDIA 的完整 CUDA 工具包（非 cudatoolkit conda 包）包含一个名为 cuda-memcheck 的实用程序，该程序可以检查 CUDA 代码中的各类内存访问错误。

让我们调试以下代码：

```
In [ ]: ! cat debug/ex3.py
```

```
In [ ]: ! cuda-memcheck python debug/ex3.py
```

cuda-memcheck 的输出明确展示了直方图函数的问题：

```
===== Invalid __global__ write of size 4
===== at 0x00000548 in cudapy::__main__:histogram$241(Array<float, int=1, C, mutable, aligned>,
float, float, Array<int, int=1, C, mutable, aligned>)
```

但我们不知道这是哪一行代码。为获得更详细的错误信息，我们可更改核函数（如下所示），进而在编译此核函数时开启“调试”模式：

```
@cuda.jit(debug=True)
def histogram(x, xmin, xmax, histogram_out):
    nbins = histogram_out.shape[0]
```

```
In [ ]: ! cuda-memcheck python debug/ex3a.py
```

现在，我们获得一则错误消息 ex3a.py:17，其中包含源文件和行号。

```
In [ ]: ! cat -n debug/ex3a.py | grep -C 2 "17"
```

此时，我们可能会发现，if 语句误用了 or，而正确用法应为 and。

cuda-memcheck 可使用不同模式来检测各类问题（与使用 valgrind 调试 CPU 内存访问错误类似）。如需了解更多信息，请访问 <http://docs.nvidia.com/cuda/cuda-memcheck/>，查阅相关文档 (<http://docs.nvidia.com/cuda/cuda-memcheck/>，查阅相关文档)。

附录：CUDA 参考资料

建议您收藏《CUDA C 语言编程指南》的第 1 章和第 2 章，以便在完成本课程后进行深入学习。该书针对 CUDA C 语言而编写，但对 CUDA Python 编程也十分适用。

- 简介: <http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#introduction> (<http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#introduction>)
- 编程模型: <http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#programming-model> (<http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#programming-model>)

附录：利用 Numba 在 GPU 上生成随机数

对于需要使用大量随机数的蒙特卡罗应用程序而言，GPU 尤为有用。CUDA 在 cuRAND 库中提供了一套出色的随机数生成算法。不过很遗憾，cuRAND 是在一组 C 头文件中进行定义的，而 Numba 却无法轻松编译或链接至此类文件（Numba 的 CUDA 即时编译器 (JIT) 从不会为 CUDA 核函数创建 C 代码。）您可以在 Numba 路线图上找到此问题的解决方案，但可能要耗费一些时间。

与此同时，Numba 0.33 版及以上版本中将包含 xoroshiro128+ 生成器。该生成器质量颇高，但相较于 cuRAND 中的 XORWOW 生成器，其周期更短 ($2^{128} - 1$)。

如要使用此生成器，您需在主机上为核函数中的每个线程初始化随机数生成器 (RNG) 状态。这种状态创建函数会按种子的指示将每个状态初始化为相同序列，但这种序列会由 2^{64} 个步长进行分隔，这样便能避免不同线程最终意外生成重叠序列（除非单线程会抽取 2^{64} 个随机数，而您绝没有耐心等待此

```
In [19]: import numpy as np
from numba import cuda
from numba.cuda.random import create_xoroshiro128p_states, xoroshiro128p_uniform_float32

threads_per_block = 64
blocks = 24
rng_states = create_xoroshiro128p_states(threads_per_block * blocks, seed=1)
```

In []:

我们可以将这些随机数状态作为参数传递至核函数，进而在函数中使用它们：

```
In [20]: @cuda.jit
def monte_carlo_mean(rng_states, iterations, out):
    thread_id = cuda.grid(1)
    total = 0
    for i in range(iterations):
        sample = xoroshiro128p_uniform_float32(rng_states, thread_id) # Returns a float32 in range [0.0, 1.0)
        total += sample

    out[thread_id] = total/iterations
```

In []:

```
In [21]: out = cuda.device_array(threads_per_block * blocks, dtype=np.float32)
monte_carlo_mean[blocks, threads_per_block](rng_states, 10000, out)
print(out.copy_to_host().mean())
```

0.50000983

In []:

练习：在 GPU 上利用蒙特卡罗法生成圆周率

让我们重温第一节中利用蒙特卡罗法生成圆周率的算法，彼时我们在 CPU 上用 Numba 编译了该算法。

```
In [22]: from numba import njit
import random

@njit
def monte_carlo_pi(nsamples):
    acc = 0
    for i in range(nsamples):
        x = random.random()
        y = random.random()
        if (x**2 + y**2) < 1.0:
            acc += 1
    return 4.0 * acc / nsamples
```

```
In [23]: nsamples = 10000000
%timeit monte_carlo_pi(nsamples)
```

126 ms ± 75.2 μs per loop (mean ± std. dev. of 7 runs, 10 loops each)

您的任务是重构下方的 monte_carlo_pi_device（目前与上方的 monte_carlo_pi 相同），使其在 GPU 上运行。您可以借鉴上方的 monte_carlo_mean，但要至少完成以下任务：

- 将其装饰为 CUDA 核函数
- 从设备 RNG 状态中为线程抽取样本（生成如下 2 个单元）
- 在输出数组中存储每个线程的结果，之后在主机上求取均值（如上方 monte_carlo_mean 所示）

查看下方两个单元后，您将发现所有数据均已初始化、执行配置已创建完成，且核函数也已启动。您只需立即重构下方单元中的核函数定义即可。如您遇到问题，请参阅 [此解决方案](#) ([./edit/solutions/monte_carlo_pi_solution.py](#))。

```
In [24]: from numba import njit
import random

# TODO: All your work will be in this cell. Refactor to run on the device successfully given the way the
# kernel is launched below.
'''
@njit
def monte_carlo_pi_device(nsamples):
    acc = 0
    for i in range(nsamples):
        x = random.random()
        y = random.random()
        if (x**2 + y**2) < 1.0:
            acc += 1
    return 4.0 * acc / nsamples
'''

'''
重构下方的 monte_carlo_pi_device (目前与上方的 monte_carlo_pi 相同), 使其在 GPU 上运行。您可以借鉴上方的 monte_carlo_mean,
将其装饰为 CUDA 核函数
从设备 RNG 状态中为线程抽取样本 (生成如下 2 个单元)
在输出数组中存储每个线程的结果, 之后在主机上求取均值 (如上方 monte_carlo_mean 所示)
'''

@cuda.jit
def monte_carlo_pi_device(rng_states, nsamples, out):
    thread_id = cuda.grid(1)

    # Compute pi by drawing random (x, y) points and finding what
    # fraction lie inside a unit circle
    acc = 0
    for i in range(nsamples):
        x = xoroshiro128p_uniform_float32(rng_states, thread_id)
        y = xoroshiro128p_uniform_float32(rng_states, thread_id)
        if x**2 + y**2 <= 1.0:
            acc += 1

    out[thread_id] = 4.0 * acc / nsamples
```

```
In [25]: # Do not change any of the values in this cell
nsamples = 10000000
threads_per_block = 128
blocks = 32

grid_size = threads_per_block * blocks
samples_per_thread = int(nsamples / grid_size) # Each thread only needs to work on a fraction of total number of
                                                # This could also be calculated inside the kernel definition using

rng_states = create_xoroshiro128p_states(grid_size, seed=1)
d_out = cuda.device_array(threads_per_block * blocks, dtype=np.float32)
```

```
In [26]: %time monte_carlo_pi_device[blocks, threads_per_block](rng_states, samples_per_thread, d_out); cuda.synchronize()

CPU times: user 130 ms, sys: 4.43 ms, total: 135 ms
Wall time: 134 ms
```

```
In [27]: print(d_out.copy_to_host().mean())

3.1415372
```

```
In [ ]:
```



DEEP
LEARNING
INSTITUTE

(<https://www.nvidia.com/dli>)

